

Unidad 2: Programación Orientada a Objetos aplicada a PHP

Aplicando los conceptos teóricos

Clase

- Definimos una clase en un archivo PHP que deberá llamarse con el mismo nombre de la clase.
- Nombre de clase en formato PascalCase.
- Palabra reservada: `class`
- El contenido de la clase se delimita con llaves `{ }`

Ejemplo: archivo **Libro.php**

```
<?php
```

```
class Libro {  
    // Atributos  
    // Métodos  
}
```

Atributos

- Se definen como variables internas de la clase.
- Nombre de atributo en formato `camelCase`.
- Agregamos los modificadores de visibilidad correspondientes:
 - `public`
 - `private`
 - `protected`
- Opcional (pero preferible): agregamos a cada atributo su tipo de dato correspondiente.

Atributos de la clase Libro

```
<?php
```

```
class Libro {  
    private string $titulo;  
    private string $autor;  
    private int $isbn;  
}
```

Métodos

- Se definen como funciones internas de la clase.
- Nombre de método en formato camelCase.
- Agregamos los modificadores de visibilidad correspondientes:
 - public
 - private
 - protected
- Opcional (pero preferible): agregamos a cada método su tipo de dato de retorno, o void si no devuelve un dato. Cada argumento también debería definir el tipo de dato.

Métodos de la clase Libro

```
<?php
```

```
class Libro {
```

```
// Atributos
```

```
public function mostrarInformacion(): void {  
    echo “El libro se llama: {$this->titulo}”;  
}  
}
```

¿Qué es `$this`?

- Dentro de una clase, la variable `$this` hace referencia a los objetos que serán creados mediante la clase.
- Con `$this` podemos acceder dentro de la clase a sus propios atributos y métodos.

Método constructor

- Método que define qué operaciones se realizan al instanciar un objeto de la clase.
- Se define con el nombre `__construct` y puede recibir argumentos.
- Se suele definir como el primer método de la clase, luego de los atributos.

Método constructor de Libro

```
<?php
```

```
class Libro {  
    public function __construct(string $titulo, string  
$autor, int $isbn) {  
        $this->titulo = titulo;  
        $this->autor = $autor;  
        $this->isbn = $isbn;  
    }  
}
```

Métodos getters y setters

- **Getters:** encargados de obtener los valores de cada atributo. Se definen con get + nombre del atributo (para booleanos: is + nombre del atributo).
- **Setters:** encargados de modificar los valores de cada atributo. Se definen con set + nombre del atributo.
- Tipos de datos de retorno:
 - getters: el tipo de dato del atributo
 - setters: void
- Se acostumbra a definirlos luego del constructor.

Getters y setters de Libro (1 ejemplo)

```
<?php
```

```
class Libro {  
    public function getTitulo(): string {  
        return $this->titulo;  
    }  
    public function setTitulo(string $titulo): void {  
        $this->titulo = $titulo;  
    }  
}
```

Instanciación

- **Instanciación:** creación de un nuevo objeto.
- Para hacerlo utilizamos la sentencia **new** + el nombre de la clase con paréntesis (y los argumentos necesarios).
- **new** utiliza el constructor de fondo.
- La variable asignada será un nuevo **objeto** o **instancia** de la clase.
- Sobre dicho objeto podemos acceder a todos los atributos y métodos públicos definidos en la clase.

Ejemplo de objetos de Libro

```
<?php
```

```
$libroOrwell = new Libro("1984", "George Orwell",  
1234567890);
```

```
$ libroOrwell->mostrarInformacion();
```

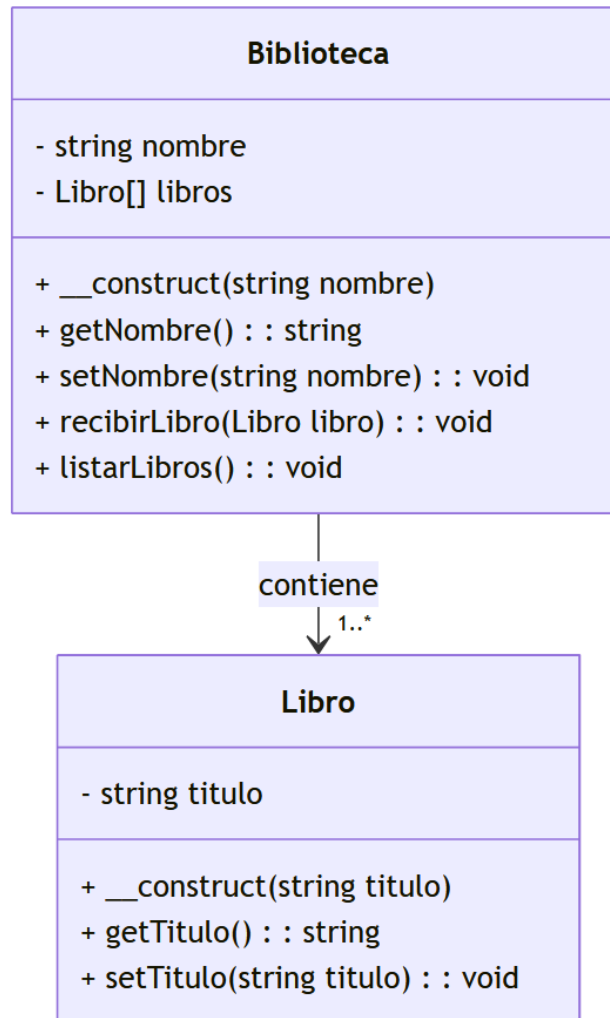
```
// El libro se llama 1984
```

```
$ libroOrwell->setTitulo("Rebelion en la granja");
```

```
$ libroOrwell->mostrarInformacion();
```

```
// El libro se llama Rebelion en la granja
```

Codificación de diagrama de clase UML



Namespaces

- Son espacios o direcciones virtuales para las clases, permitiendo organizar de mejor forma el código.
- Utilidad:
 - Evita la colisión de clases que se llamen de la misma forma.
 - Organiza el código de un proyecto en base la naturaleza del mismo.
 - Facilita el proceso de autoload (autocarga de clases) en proyectos de mayor complejidad.
 - Permite incluir solo los elementos necesarios de un archivo.
- Los namespaces no necesariamente deben seguir la estructura de carpetas, pero es lo recomendado.

Ejemplo

- Al inicio de una clase, declaramos a que namespace pertenece:

```
namespace Modelos;  
class Usuario {  
    // Código  
}
```

- Para usarla en otro archivo, cargamos el archivo y utilizamos la sentencia use + el namespace completo de la clase:

```
require_once 'Modelos/Usuario.php';  
use Modelos\Usuario;  
$usuario = new Usuario();
```

Ejemplo: colisión de nombres

✓ antiguo

🐘 Libro.php

🐘 Biblioteca.php

🐘 index.php

🐘 Libro.php

Ejercicio 1

- Tomando el siguiente diagrama de clase UML, crea las clases y la relación en PHP.
- Agrega los namespaces correspondientes.
- Crea tres estudiantes y agregalos a un curso nuevo en un tercer archivo.
- Muestra en pantalla los nombres y legajos de todos los estudiantes del curso.

